



CPSC 100

Computational Thinking

Intro to Programming

Instructor: Firas Moosvi
Department of Computer Science
University of British Columbia



Agenda

- Course Admin
- Course Project
- Learning Goals
 - Components of an Algorithm
 - Conditionals and Loops in Snap!

Course Admin



CPSC 100 Term Project

- The goal of the CPSC 100 term project is for you to create a “digital artifact”: an **infographic/video/learning with lyrics!** to help explain a topic that touches on several parts of the CPSC 100 course.



CPSC 100 Term Project

- Project component is 25% of your CPSC 100 grade
- There will be 5 deliverables called “Milestones”
 - NOT 5% each, last one will be worth the most!
 - Milestones will build on top of each other
 - Feedback will be given for each milestone to help you improve
- Projects will be done in groups of 3 or 4, **groups will be formed in class (soon)**
 - **There will be group *and* individual components**



CPSC 100 Term Project

Milestones	Purpose
Milestone 1	Group Formation & Team Contract
Milestone 2	Proposal & Sketch
Milestone 3	Individual Contributions
Milestone 4	Group Contribution
Milestone 5	Final Presentation & Peer Review



Learning Goals

After this week's lectures, you should be able to:

- Use logical reasoning to **determine the output of given code**
- Apply CT to **trace and evaluate** code snippets
- Define **boolean expressions** and their role in Snap! Programming
- **Apply** boolean operations to control flow in programming
- Describe the concept of **loops** and **iteration** in programming
 - Differentiate between **finite loops** (repeat N times) and **condition-based loops** (repeat until)

Bread Making Algorithm

Repeat 10 times:

1. Preheat oven (400° C)
2. Combine ingredients in bowl to form dough
3. Put dough into bread pan
4. If ingredients contain yeast, allow to sit at room temperature for 1 hour
5. Put bread pans into preheated oven and bake for 30 minutes

Task: In this algorithm, where do you see the three components?

Components of an Algorithm

1. Sequencing
2. Selection
3. Iteration

Components of an Algorithm

1. Sequencing
2. Selection
3. Iteration



Sequencing

Instructions are executed in the specified order

A large, thick purple arrow points vertically downwards along the left side of the slide, starting from the level of the 'Instructions are executed...' text and extending past the bottom of the list.

Repeat 10 times:

1. Preheat oven (400° C)
2. Combine ingredients in bowl to form dough
3. Put dough into bread pan
4. If ingredients contain yeast, allow to sit at room temperature for 1 hour
5. Put bread pans into preheated oven and bake for 30 minutes

Q: What is the value of “age” after all lines of this program are executed?



- A. 1
- B. 40
- C. 41
- D. 401

```

when clicked
  set age to 40
  say join Your age is now age for 2 secs
  set age to age + 1
  say join Happy Birthday! Your age is now age for 2 secs
  
```

Components of an Algorithm

- ~~1. Sequencing~~
- 2. Selection**
3. Iteration



Selection

Allows the algorithm to select which instructions to execute
(depending on conditions)

A large, thick purple arrow points vertically downwards along the left side of the slide, starting from the level of the 'Selection' header and extending past the bottom of the list.

Repeat 10 times:

1. Preheat oven (400° C)
2. Combine ingredients in bowl to form dough
3. Put dough into bread pan
4. **If ingredients contain yeast, allow to sit at room temperature for 1 hour**
5. Put bread pans into preheated oven and bake for 30 minutes



Selection → Conditionals

Conditionals allow for different results, depending on input. Generally this looks like “**if**” with a possible “**else**”:

Real World:

If you eat your dinner

Then you may have some ice cream

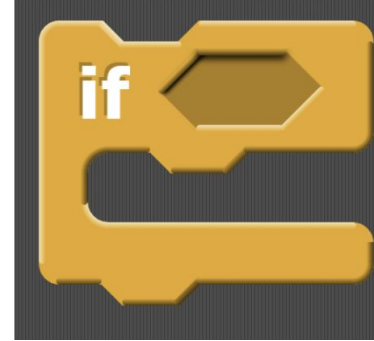
Else (you may have referred to this step as "otherwise" in Lab 2)

You may only have fruit



Selection → Conditionals

If ingredients contains yeast
allow to sit at room temp. for 1hr



If it's snowing
Wake-up 6:30am

Else
Wake-up at 7am



Q: What's the output after we press run?



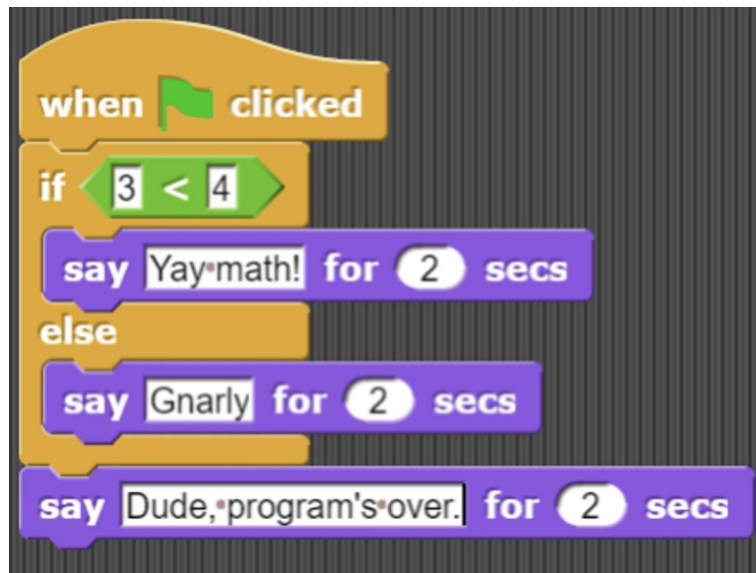
iClicker

A. It only says “Yay math!”

B. It says “Yay math!” Then...
it says “Gnarly” Then...
it says “Dude, program’s over.”

C. It says “Gnarly” Then...
it says “Dude, program’s over.”

D. It says something else

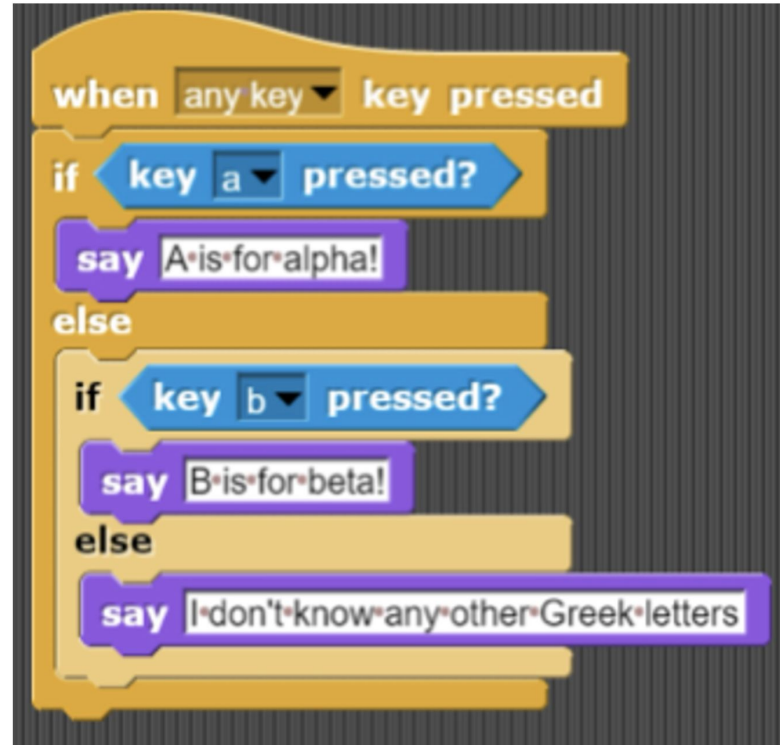


Q: What is the output if you press the “P” key?

- A. “A is for alpha”
- B. “B is for beta”
- C. “I don’t know any other Greek letters”
- D. The program crashes
- E. No output



iClicker



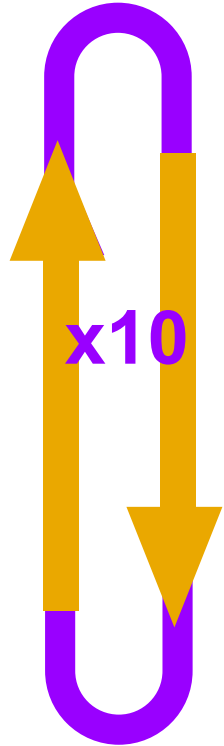


Components of an Algorithm

- ~~1. Sequencing~~
- ~~2. Selection~~
- 3. Iteration**

Iteration

Allows the algorithm to repeat instructions.



Repeat 10 times:

1. Preheat oven (400° C)
2. Combine ingredients in bowl to form dough
3. Put dough into bread pan
4. If ingredients contain yeast, allow to sit at room temperature for 1 hour
5. Put bread pans into preheated oven and bake for 30 minutes

7:35 69%

← Otis Redding

Music Merch

you liked
1 song • Otis Redding

Popular

- 1 (Sittin' On) the Dock of the Bay 762,315,950
- 2 Hard to Handle 127,583,692
- 3 These Arms of Mine 124,408,002
- 4 Stand by Me 251,400,923
- 5 ... Try a Little Tenderness 148,756,793

Artist Pick

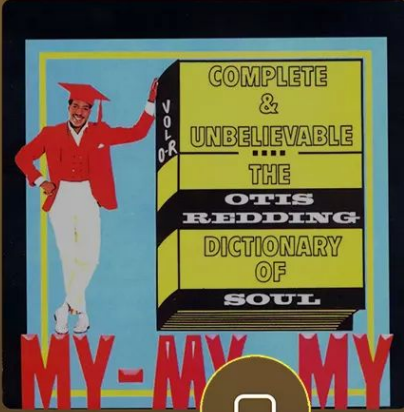
Posted by Otis Redding

 Try a Little Tenderness
Otis Redding

Home Search Your Library

7:11 72%

60s Mix



Try a Little Tenderness
Otis Redding

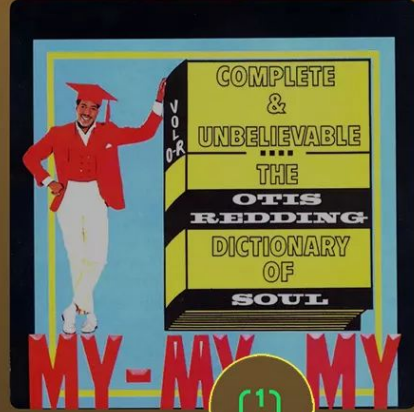
1:59 -1:22

⏮ ⏪ ⏩ ⏭

Lyrics

7:11 72%

60s Mix



Try a Little Tenderness
Otis Redding

1:59 -1:22

⏮ ⏪ ⏩ ⏭

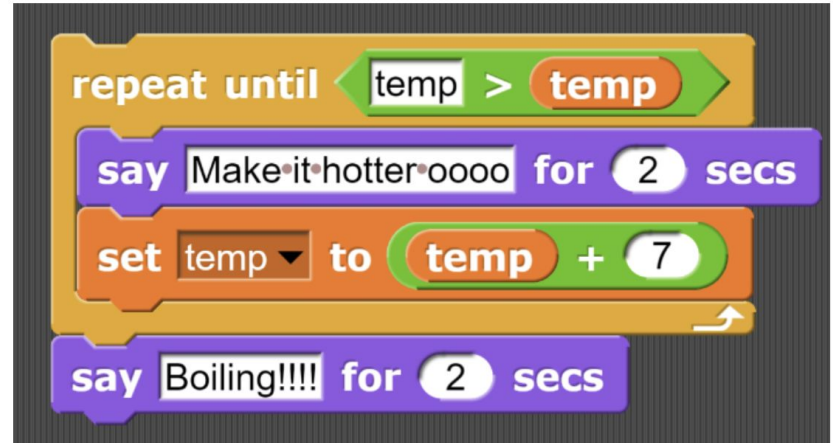
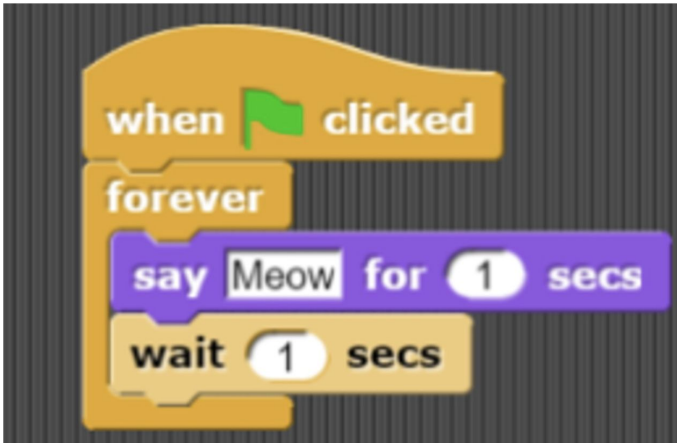
Lyrics



Iteration

What if you want to do a task over and over again?

A **loop** allows you to do the same task over & over again, sometimes with a **stopping** condition, sometimes **forever**!



Repeat Blocks

Repeat some code a finite number of time



Repeat UNTIL a particular condition has been met.

If the condition is never met, then, it goes on *forever*.

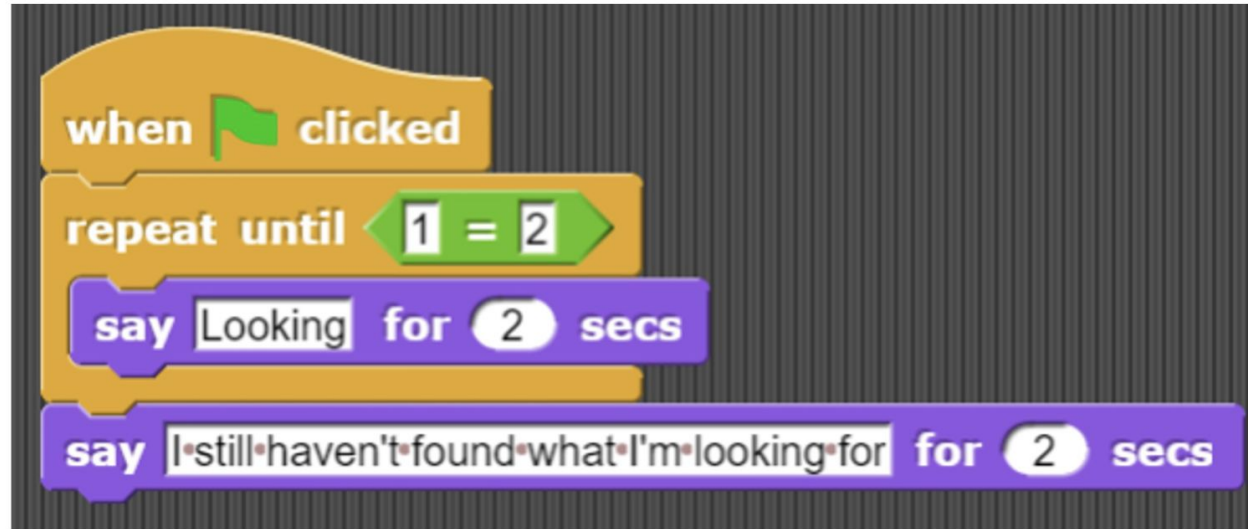




Q: Will this program ever say “I still haven’t found what I’m looking for”?



- A. Yes
- B. No
- C. Sometimes



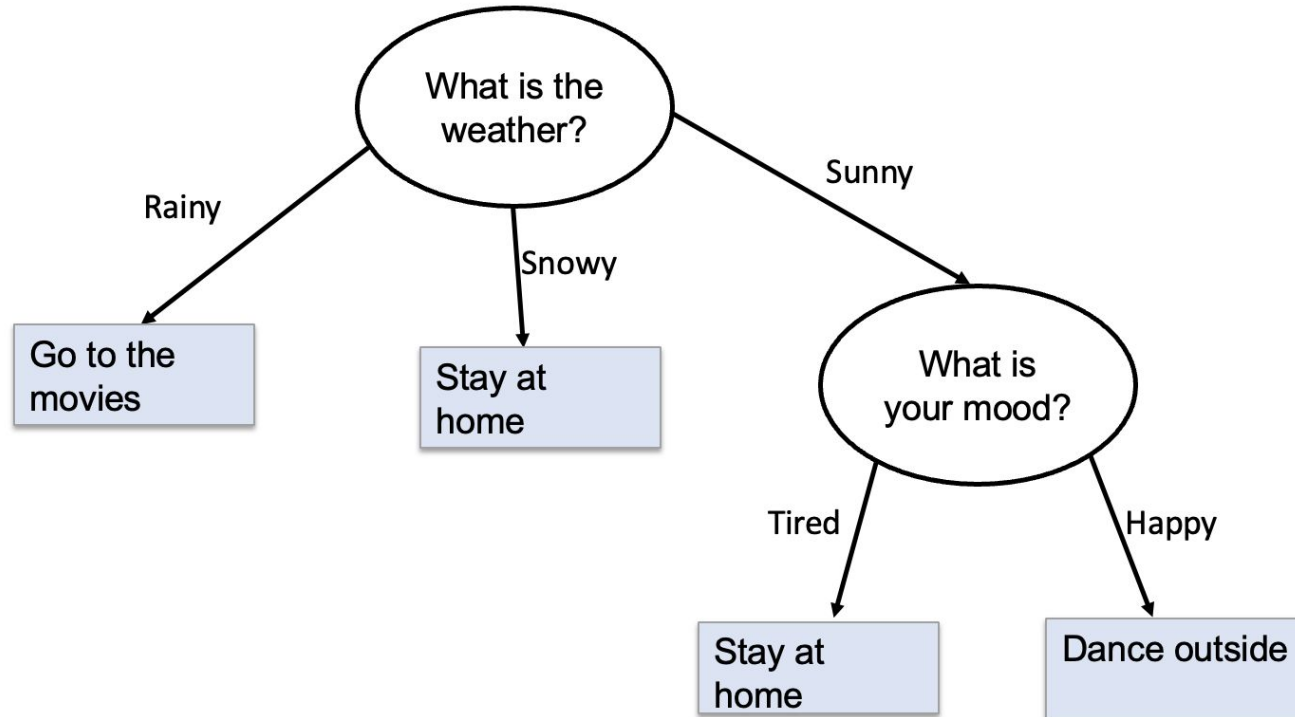


Class Activity



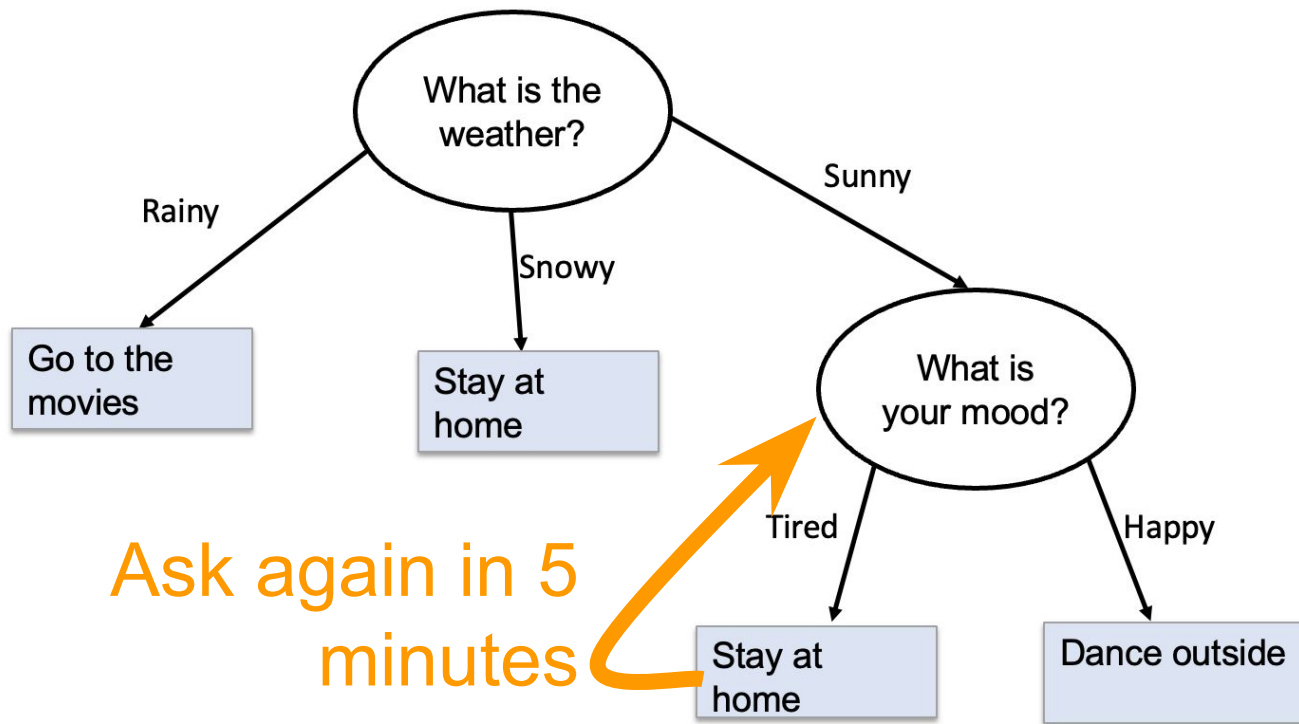
A. Conditionals in Snap!

Convert the following decision tree to a Snap Block program, submit a picture of your scripting panel on PrairieLearn



B. Loops in Snap!

Convert the following decision tree to a Snap Block program, submit a picture of your scripting panel on PrairieLearn



Wrap up

Boolean (or Logical) Function



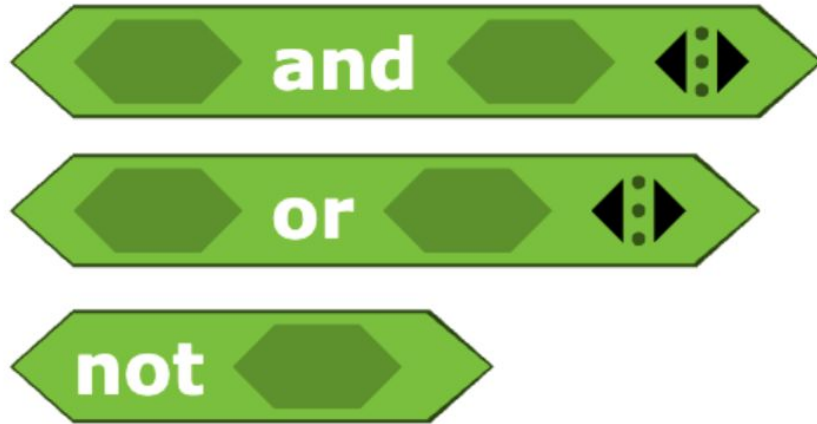
Boolean (or Logical) Function

At the very lowest level, computer circuitry is made of wires, and each wire either has power or it does not: meaning it's either **on or off**.

The only operations that can be performed at the lowest level are those that operate on **single-bit values**

0 or 1 on or off

Boolean (or Logical) Function



Notice that both the blocks themselves and the input slots in the blocks are hexagonal.

Boolean functions take Boolean values (True or False) as **inputs** and report a new Boolean value as **output**.

Boolean → Data Representation

OR Blocks | Evaluates to

 False

 True

 True

 True

OR evaluates to **true**, as long as **one operand evaluates to true**

Boolean → Data Representation

OR Blocks | Evaluates to

 False

 True

 True

 True

AND Blocks | Evaluates to

 False

 False

 False

 True

OR evaluates to **true**, as long as one operand evaluates to true

AND evaluates to **true**, only if **all** operands are **true**

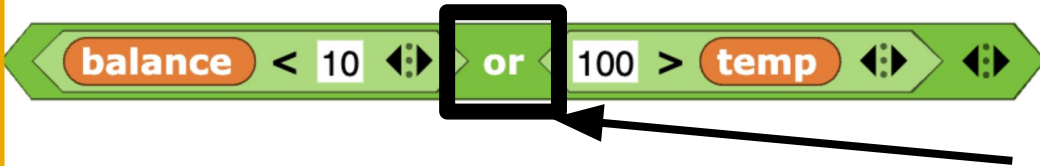
Boolean Function Examples



balance and temp are variables



if balance is less than 10 **AND**
if temp is greater than 100



if balance is less than 10 **OR**
if temp is greater than 100



if balance is **NOT** less than 10

For loops

when  clicked

for **i** = 1 to 10

say **i** for 1 secs

say Bye-bye! for 2 secs

when  clicked

for  = 1 to 10

say  for 1 secs

say Bye-bye! for 2 secs

i is a variable

when  clicked

for  = 1 to 10

say  for 1 secs

say Bye-bye! for 2 secs

**i is initialized
with a value of 1**

when  clicked

for **i** = 1 to 10

say **i** for 1 secs

say Bye-bye! for 2 secs

This loop will run until i has a value that is not between 1 to 10 (inclusive)

when  clicked

for **i** = 1 to 10

say **i** for 1 secs

say Bye-bye! for 2 secs

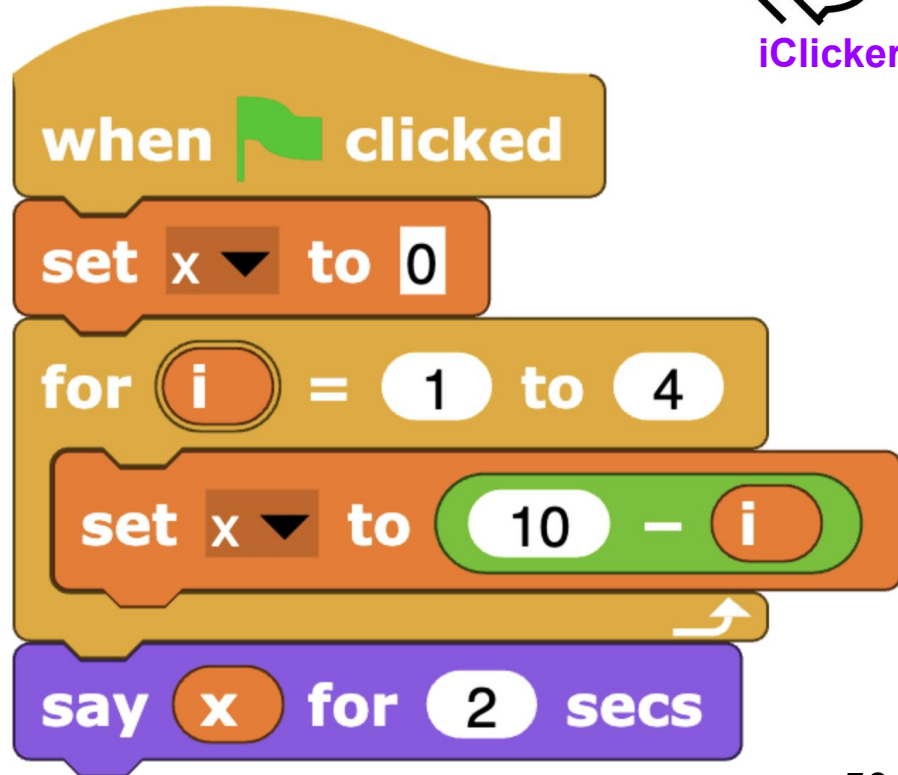
Every time we reach the end of the loop, i will increase (increment) by 1

Q: What is the value in x when the code is run?



iClicker

- A. 1
- B. 3
- C. 4
- D. 6
- E. 10



Q: What is the value in **x** when the code is run?



iClicker

- A. 1
- B. 3
- C. 4
- D. 6
- E. 10

